

4. Liskov Substitution Principle (LSP) in Java

4.1 Introduction

The **Liskov Substitution Principle (LSP)** is the third **SOLID** principle. It ensures that subclasses remain interchangeable with their parent classes **without altering the correctness** of the program.

4.2 Problem It Solves

Prevents unexpected behavior or bugs when a child class overrides a parent method in a way that violates assumptions made by the client.

4.3 Definition of LSP

"Subtypes must be substitutable for their base types."

In simpler terms: wherever a parent class object is expected, you should be able to use a child class object without breaking functionality.

4.4 Real-World Analogy

If a credit card is a type of payment method, any subclass like Visa, MasterCard should work exactly the same when passed to a payment system — without changing how the system behaves.

4.5 Symptoms of LSP Violation

- Subclasses throw `UnsupportedOperationException`
- Subclasses override methods with incompatible logic
- Client code needs to check `instanceof`

4.6 Benefits of LSP

- Improves code reliability
- Simplifies testing (subclass tests also validate base contracts)
- Allows safe polymorphism and interchangeability

4.7 Implementation Guidelines

- Prefer interfaces and composition
- Follow behavioral expectations
- Do not override methods with contradictory behavior
- Avoid adding new constraints in subclasses

4.8 Examples

Example1: Violation: Shape Area Calculation

```
package com.mannemlokesh.violation.example1;

//SRP Violation
class Rectangle {
    protected int width, height;
```

```

    public void setWidth(int width) {
        this.width = width;
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

class Square extends Rectangle {
    public void setWidth(int width) {
        this.width = width;
        this.height = width;
    }

    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}

public class ShapeClient {
    static void printArea(Rectangle r) {
        r.setWidth(5);
        r.setHeight(4);
        System.out.println("Area: " + r.getArea());
    }

    public static void main(String[] args) {
        Rectangle rect = new Rectangle();
        Square square = new Square();
        printArea(rect); // Expected: 20
        printArea(square); // Unexpected: 16
    }
}

```

Output:

```

Area: 20
Area: 16

```

Example1: Compliant : Shape Area Calculation

```

package com.mannemlokes.compliant.example1;

interface Shape {
    int getArea();
}

class Rectangle implements Shape {
    private int width, height;

    public void setWidth(int width) {
        this.width = width;
    }
}

```

```
    }

    public void setHeight(int height) {
        this.height = height;
    }

    public int getArea() {
        return width * height;
    }
}

class Square implements Shape {
    private int side;

    public void setSide(int side) {
        this.side = side;
    }

    public int getArea() {
        return side * side;
    }
}

public class ShapeClientFixed {
    public static void main(String[] args) {
        Rectangle rect = new Rectangle();
        rect.setWidth(5);
        rect.setHeight(4);
        System.out.println("Rectangle Area: " + rect.getArea());

        Square square = new Square();
        square.setSide(4);
        System.out.println("Square Area: " + square.getArea());
    }
}
```

Output:

```
Rectangle Area: 20
Square Area: 16
```

Example2: Violation: Bird Inheritance Management

```
package com.mannemlokesh.violation.example2;

class Bird {
    public void fly() {
        System.out.println("Flying...");
    }
}

class Ostrich extends Bird {
    @Override
    public void fly() {
        throw new UnsupportedOperationException("Ostriches can't fly");
    }
}
```

```
public class BirdClientViolation {
    public static void main(String[] args) {
        Bird bird = new Ostrich();
        bird.fly(); // Runtime exception
    }
}
```

Example2: Compliant : Bird Inheritance Management

```
package com.mannemlokes.compliant.example2;

interface Flyable {
    void fly();
}

class Eagle implements Flyable {
    public void fly() {
        System.out.println("Eagle flying");
    }
}

class Ostrich {
    public void walk() {
        System.out.println("Ostrich walking");
    }
}

public class BirdClientFixed {
    public static void main(String[] args) {
        Flyable eagle = new Eagle();
        eagle.fly();

        Ostrich ostrich = new Ostrich();
        ostrich.walk();
    }
}
```

Output:

```
Eagle flying
Ostrich walking
```

Example3: Violation : Account Interface with LSP

```
package com.mannemlokes.violation.example3;

interface Account {
    void deposit(double amount);

    void withdraw(double amount);
}

class SavingsAccount implements Account {
    private double balance;

    public void deposit(double amount) {
```

```
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }
}

class ReadOnlyAccount implements Account {
    public void deposit(double amount) {
        throw new UnsupportedOperationException("Cannot deposit");
    }

    public void withdraw(double amount) {
        throw new UnsupportedOperationException("Cannot withdraw");
    }
}

public class AccountClient {
    public static void main(String[] args) {
        // ...
    }
}
```

Example3: Complaint : Account Interface with LSP

```
package com.mannemlokes.compliant.example3;

interface DepositAccount {
    void deposit(double amount);
}

interface WithdrawAccount {
    void withdraw(double amount);
}

class FullAccount implements DepositAccount, WithdrawAccount {
    private double balance;

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        balance -= amount;
    }
}

class AuditAccount implements DepositAccount {
    public void deposit(double amount) {
        System.out.println("Audit log: Deposit = " + amount);
    }
}

public class AccountClient {
    public static void main(String[] args) {
        FullAccount account = new FullAccount();
        account.deposit(1000);
        account.withdraw(400);
    }
}
```

```
AuditAccount audit = new AuditAccount();
audit.deposit(250);
}
```

Output:

```
Audit log: Deposit = 250.0
```

4.9 Common Pitfalls

- Forcing subclasses to override irrelevant methods
- Assuming "is-a" relationship is always safe
- Relying on runtime type checks

4.10 LSP vs Inheritance vs Polymorphism

Concept	Purpose
Inheritance	Code reuse
Polymorphism	Replace object at runtime
LSP	Ensures substitution won't break

4.11 When to Apply LSP

- When designing class hierarchies
- When using polymorphism
- When using inheritance or interfaces

4.12 Summary

Feature	Description
Intent	Subtypes can replace base types safely
Use Case	API design, polymorphism, class hierarchy
Key Benefit	More predictable, reliable inheritance

The Liskov Substitution Principle ensures that your inheritance is safe, logical, and robust. It promotes reliable code reuse and effective polymorphism.